

1704. Determine if String Halves Are Alike

Solved

EasyTopicsCompareHint

You are given a string `s` of even length. Split this string into two halves of equal lengths, and let `a` be the first half and `b` be the second half.

Two strings are **alike** if they have the same number of vowels ('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'). Notice that `y` contains uppercase and lowercase letters.

Return `true` if `a` and `b` are **alike**. Otherwise, return `false`.

Example 1:

Input: `s = "book"`
Output: `true`
Explanation: `a = "bo"` and `b = "ok"`. `a` has 1 vowel and `b` has 1 vowel. Therefore, they are alike.

Example 2:

Input: `s = "textbook"`
Output: `false`
Explanation: `a = "text"` and `b = "book"`. `a` has 1 vowel whereas `b` has 2. Therefore, they are not alike. Notice that the vowel `o` is counted twice.

Constraints:

- $2 \leq s.length \leq 1000$
- `s.length` is even.
- `s` consists of **uppercase** and **lowercase** letters.

Seen this question in a real interview before? 1/6

YesNo

2.3K108

Code

JavaAuto

```
1 class Solution {
2     public boolean halvesAreAlike(String s) {
3         String vowels = "aeiouAEIOU";
4         int n = s.length() / 2;
5         int count = 0;
6
7         for (int i = 0; i < n; i++) {
8             if (vowels.indexOf(s.charAt(i)) != -1) {
9                 count++;
10            }
11        }
12
13        for (int i = n; i < s.length(); i++) {
14            if (vowels.indexOf(s.charAt(i)) != -1) {
15                count--;
16            }
17        }
18
19        return count == 0;
20    }
21 }
22
```

Saved

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

`s = "book"`

Output

`true`

Expected

Difficulty: 1559Expand

StatementHintsSubmissionsSolutionHelp

Lapindromes

Lapindrome is defined as a string which when split in the middle, gives two halves having the same characters and same frequency of each character. If there are odd number of characters in the string, we ignore the middle character and check for lapindrome. For example `gaga` is a lapindrome, since the two halves `ga` and `ga` have the same characters with same frequency. Also, `abccab`, `rotor` and `xyzyx` are a few examples of lapindromes. Note that `abbaab` is NOT a lapindrome. The two halves contain the same characters but their frequencies do not match.

Your task is simple. Given a string, you need to tell if it is a lapindrome.

Input:

First line of input contains a single integer `T`, the number of test cases.

Each test is a single line containing a string `S` composed of only lowercase English alphabet.

Output:

For each test case, output on a separate line: "YES" if the string is a lapindrome and "NO" if it is not.

Constraints:

- $1 \leq T \leq 100$
- $2 \leq |S| \leq 1000$, where `|S|` denotes the length of `S`

Sample 1:

Input	Output
6	YES
gaga	NO
abccde	YES
rotor	YES
xyzyx	NO
abbaab	NO

Java

```
1 import java.util.*;
2 import java.lang.*;
3 import java.io.*;
4 class Codechef {
5     public static void main (String[] args) throws java.lang.Exception {
6         Scanner sc = new Scanner(System.in);
7         int T = sc.nextInt();
8         while (T-- > 0) {
9             String s = sc.next();
10            int n = s.length();
11            int[] left = new int[26];
12            int[] right = new int[26];
13            int half = n / 2;
14            for (int i = 0; i < half; i++) {
15                left[s.charAt(i) - 'a']++;
16            }
17            int start = (n % 2 == 0) ? half : half + 1;
18            for (int i = start; i < n; i++) {
19                right[s.charAt(i) - 'a']++;
20            }
21            if (Arrays.equals(left, right)) {
22                System.out.println("YES");
23            } else {
24                System.out.println("NO");
25            }
26        }
27        sc.close();
28    }
29 }
```

Test against Custom Input

Correct AnswerSubmission ID: 1348120655

Sub-Task	Task #	Result (time)
		Accepted

Visualize CodeRunSubmit

Problem List

217. Contains Duplicate

Solved

EasyTopicsCompare

Given an integer array `nums`, return `true` if any value appears at **least twice** in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`
Output: `true`
Explanation:
The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`
Output: `false`
Explanation:
All elements are distinct.

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`
Output: `true`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

14.2K469

Code

JavaAuto

```
1 import java.util.*;
2
3 class Solution {
4     public boolean containsDuplicate(int[] nums) {
5         HashSet<Integer> set = new HashSet<>();
6
7         for (int num : nums) {
8             if (set.contains(num)) {
9                 return true;
10            }
11            set.add(num);
12        }
13
14        return false;
15    }
16 }
17
```

Saved

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3

Input

`nums = [1,2,3,1]`

Output

`true`

Expected

Problem List

867. Transpose Matrix

Easy

Topics

Companies

Hint

Given a 2D integer array `matrix`, return the transpose of `matrix`.

The **transpose** of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.

2

4

-1

-10

5

11

18

-7

6

2

-10

18

4

5

-7

-1

11

6

Example 1:

Input: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`

Output: `[[1,4,7],[2,5,8],[3,6,9]]`

Example 2:

Input: `matrix = [[1,2,3],[4,5,6]]`

Output: `[[1,4],[2,5],[3,6]]`

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 1000`
- `-109 <= matrix[i][j] <= 109`

4.3K 133 17 Online

Code

Java

Auto

```
1 class Solution {
2     public int[][] transpose(int[][] matrix) {
3         // Get the dimensions of the original matrix
4         int m = matrix.length;
5         int n = matrix[0].length;
6         // Create a new matrix with inverted dimensions
7         int[][] result = new int[n][m];
8
9         // Map the elements from the original matrix to the transposed matrix
10        for (int i = 0; i < m; i++) {
11            for (int j = 0; j < n; j++) {
12                result[j][i] = matrix[i][j];
13            }
14        }
15        return result;
16    }
17 }
```

Testcase 1 Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`matrix = [[1,2,3],[4,5,6],[7,8,9]]`

Output

`[[1,4,7],[2,5,8],[3,6,9]]`

1/30, Col 1

Search...

Courses

Tutorials

Practice

Jobs

Sign In

Problem

Editorial

Submissions

Comments

Difficulty: Medium

Accuracy: 54.72%

Submissions: 22K+

Points: 4

Average Time: 15m

Multiply 2 Matrices

Given two square matrices `mat1` and `mat2`, each of size `n × n`. Multiply these two matrices and return the resulting matrix.

Examples:

Input: `mat1 = [[1, 1, 1], [1, 1, 1], [1, 1, 1]]`, `mat2 = [[1, 1, 1], [1, 1, 1], [1, 1, 1]]`

Output: `[[3, 3, 3], [3, 3, 3], [3, 3, 3]]`

Explanation: After multiplying `mat1` and `mat2` we get the resulting matrix equal to `[[3, 3, 3], [3, 3, 3], [3, 3, 3]]`.

Input: `mat1 = [[1, 2], [3, 4]]`, `mat2 = [[4, 3], [2, 1]]`

Output: `[[8, 5], [20, 13]]`

Explanation: After multiplying `mat1` and `mat2` we get the resulting matrix equal to `[[8, 5], [20, 13]]`.

Constraints:

- `1 ≤ n ≤ 100`
- `1 ≤ mat1[i][j], mat2[i][j] ≤ 100`

Code

Java

211

```
1 class Solution {
2     public int[][] multiplyMatrix(int[][] mat1, int[][] mat2) {
3         int n = mat1.length;
4         int[][] result = new int[n][n];
5
6         // Iterate through each row of mat1
7         for (int i = 0; i < n; i++) {
8             // Iterate through each column of mat2
9             for (int j = 0; j < n; j++) {
10                // Compute the dot product of row i and column j
11                for (int k = 0; k < n; k++) {
12                    result[i][j] += mat1[i][k] * mat2[k][j];
13                }
14            }
15        }
16        return result;
17    }
18 }
```

Testcase 1 Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`mat1 = [[1, 2], [3, 4]]`

mat2 = [[4, 3], [2, 1]]

Output

`[[8, 5], [20, 13]]`

1/30, Col 1

Problem List

1314. Matrix Block Sum

Medium

Topics

Companies

Hint

Given a `m × n` matrix `mat` and an integer `k`, return a matrix `answer` where each `answer[i][j]` is the sum of all elements `mat[r][c]` for:

- `0 ≤ r < m`, `0 ≤ c < n`, and
- `0 ≤ r1 < r2 < m`, `0 ≤ c1 < c2 < n`, and
- `(r1, c1)` is a valid position in the matrix.

Example 1:

Input: `mat = [[1,2,3],[4,5,6],[7,8,9]]`, `k = 1`

Output: `[[12,21,16],[27,45,33],[24,39,28]]`

Example 2:

Input: `mat = [[1,2,3],[4,5,6],[7,8,9]]`, `k = 2`

Output: `[[45,45,45],[45,45,45],[45,45,45]]`

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 ≤ m, n, k ≤ 100`
- `1 ≤ mat[i][j] ≤ 100`

Discover more Skill Assessment Tools

2.6K 12 27 Online

Code

Java

Auto

```
1 class Solution {
2     public int[][] matrixBlockSum(int[][] mat, int k) {
3         int m = mat.length;
4         int n = mat[0].length;
5
6         // Step 1: Build the 2D prefix sum matrix
7         // The prefix matrix is padded with an extra row and column (size m+1 x n+1)
8         // to handle boundary conditions easily without extra if-else checks.
9         int[][] pref = new int[m+1][n+1];
10        for (int i = 0; i < m; i++) {
11            for (int j = 0; j < n; j++) {
12                pref[i+1][j+1] = pref[i+1][j] + pref[i][j+1] + mat[i][j];
13            }
14        }
15
16        // Step 2: Calculate the block sum for each cell
17        int[][] ans = new int[m][n];
18        for (int i = 0; i < m; i++) {
19            for (int j = 0; j < n; j++) {
20                // Define the bounding box constraints, keeping within matrix indices
21                int r1 = Math.max(0, i - k);
22                int r2 = Math.min(m - 1, i + k);
23                int c1 = Math.max(0, j - k);
24                int c2 = Math.min(n - 1, j + k);
25
26                // Use the 2D prefix sum formula (Inclusion-Exclusion Principle)
27                ans[i][j] = pref[r2+1][c2+1] - pref[r1+1][c2+1] - pref[r2+1][c1] + pref[r1+1][c1];
28            }
29        }
30    }
31 }
```

Testcase 1 Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`mat = [[1,2,3],[4,5,6],[7,8,9]]`

k = 1

Output

`[[12,21,16],[27,45,33],[24,39,28]]`

1/30, Col 1

Matrix Layer Rotation

Problem

Submissions

Leaderboard

Discussions

Editorial

You are given a 2D matrix of dimension `m × n` and a positive integer `k`. You have to rotate the matrix `k` times and print the resultant matrix. Rotation should be in anti-clockwise direction.

Rotation of a `4 × 5` matrix is represented by the following figure. Note that in one rotation, you have to shift elements by one step only.

$a_{1,1}$

$a_{2,1}$

$a_{3,1}$

$a_{4,1}$

$a_{1,2}$

$a_{2,2}$

$a_{3,2}$

$a_{4,2}$

$a_{1,3}$

$a_{2,3}$

$a_{3,3}$

$a_{4,3}$

$a_{1,4}$

$a_{2,4}$

$a_{3,4}$

$a_{4,4}$

$a_{1,5}$

$a_{2,5}$

$a_{3,5}$

$a_{4,5}$

Matrix Rotation

It is guaranteed that the minimum of `m` and `n` will be even.

As an example rotate the Start matrix by 2:

Start

First

Second

1 2 3 4

12 1 2 5

11 4 3 6

10 9 8 7

→

→

→

2 3 4 5

1 2 3 6

12 1 4 7

11 10 9 8

→

→

→

3 4 5 6

2 3 4 7

1 2 1 8

12 11 10 9

→

→

→

Function Description

